Django REST Framework

Total Time: 3 Hours

Theme: Doctor Finder API

Section A: Conceptual Understanding

1. Explain why REST APIs are preferred for mobile and frontend-heavy apps.
2. Explain how serializers act as a validation layer and how to implement custom field-level validation.
3. Explain the importance of using appropriate HTTP status codes (201 Created vs 200 OK) for API outcomes.
4. Explain why pagination is required when listing doctors and how it impacts database performance.
5. Explain the benefits of ViewSets over APIView for rapid CRUD development.
6. Explain the use of Atomic Transactions (transaction.atomic) when creating related doctor records to ensure data integrity.

Section B: Practical Tasks

1. **Doctor Model Definition:** Create a `Doctor` model in `models.py` with fields for `name` (`CharField`), `specialization` (`CharField`), and `city` (`CharField`). Ensure you run migrations to update your database schema.
2. **CRUD APIs with ModelViewSet & Router:** Develop a `DoctorViewSet` inheriting from `viewsets.ModelViewSet`. Register this viewset in `urls.py` using `DefaultRouter`, which automatically generates standard endpoints for **GET**, **POST**, **PUT**, and **DELETE** operations.
3. **Implement API Pagination:** Configure pagination globally in `settings.py` or locally in your viewset. Choose `PageNumberPagination` for simple page-based navigation or `LimitOffsetPagination` to allow clients to specify the exact number of records and starting point.

4. **Atomic Transaction for Creation:** Override the `perform_create` method in your viewset. Wrap the `serializer.save()` call inside a `transaction.atomic()` block. This guarantees that if any part of the record creation fails, no partial or "orphan" data is committed to the database.
5. **API Verification with Postman:** Launch **Postman** to test your endpoints. Send a **GET** request to verify the paginated list of doctors and a **POST** request with a JSON body to confirm the creation logic returns a `201 Created` status with the correct JSON payload.

Section C: Mini Project

Doctor Finder REST API

Objective: Develop a robust, production-ready API for managing medical professional data with advanced querying and data safety.

1. **Doctor Profile CRUD** Use **DRF Serializers** and **ViewSets** to handle JSON data conversion and validation. This provides standard endpoints for creating, reading, updating, and deleting doctor records with minimal boilerplate.
2. **Pagination & Ordering** Implement **LimitOffsetPagination** to handle large datasets efficiently. Add `OrderingFilter` to enable client-side sorting by fields like `name` or `specialization` via URL parameters (e.g., `?ordering=-name`).
3. **Atomic Transactions** Wrap sensitive operations in `transaction.atomic` to guarantee data integrity. This ensures that if any part of a profile update fails, the entire database operation rolls back, preventing partial or corrupt data entries.
4. **API Documentation** Utilize DRF's **Browsable API** to test endpoints directly in the browser. This allows for quick debugging of JSON structures and HTTP status codes before frontend integration.